SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Reverse Engineering Task

Code of Conduct:

*I **D Varshini /192320007 (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

Answer:

1. Reverse-engineered analysis

The legacy design behaves like a set of large modules: registration, course management, content delivery, evaluation and result generation contain both user-interface code and business rules. This produces low cohesion inside the modules and strong coupling between screens, rules and persistence logic. A small change in a registration or evaluation rule can therefore force changes in several unrelated parts of the system.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods
Student	Represents a learner and owns student-specific operations.	studentId, name, email, status	enroll(course), submit(assignment), viewResult()
Course	Represents a course offered by the system.	courseId, title, credits, instructorId	addContent(), addAssignment(), getStudents()
Enrollment	Connects a Student with a Course.	enrollmentId, enrolledDate, status	activate(), cancel()

ContentItem	Represents learning material.	contentId, title, type, url	publish(), update()
Assignment	Represents work assigned for a course.	assignmentId, dueDate, maxMarks	publish(), evaluate(submission)
Submission	Stores a student submission and mark.	submissionId, submittedAt, score	submit(), setScore()
Result	Represents final academic outcome.	resultId, total, grade, status	calculateGrade(), publish()
Service classes	Coordinate use cases without UI or DB code.	RegistrationService, CourseService, EvaluationService, ResultService	registerStudent(), createCourse(), evaluate(), generateResult()
Repository classes	Hide OODBMS access behind interfaces.	StudentRepository, CourseRepository, AssignmentRepository, ResultRepository	findById(), save(), update(), delete()

3. Relationships and redesigned structure

A Student has many Enrollments, and a Course has many Enrollments; this resolves the many-to-many Student-Course relationship. A Course aggregates ContentItems and Assignments. A Student creates Submissions for Assignments, and a Submission may produce a Result. Service classes depend on repository interfaces rather than directly on the OODBMS.

- Presentation layer: StudentUI, InstructorUI and AdminUI only collect input and display output.
- Application/service layer: coordinates registration, course, evaluation and result use cases.
- Domain layer: contains Student, Course, Enrollment, ContentItem, Assignment, Submission and Result with core business rules.
- Persistence layer: repository/DAO implementations map domain objects to the OODBMS.

4. UML class / architecture diagram

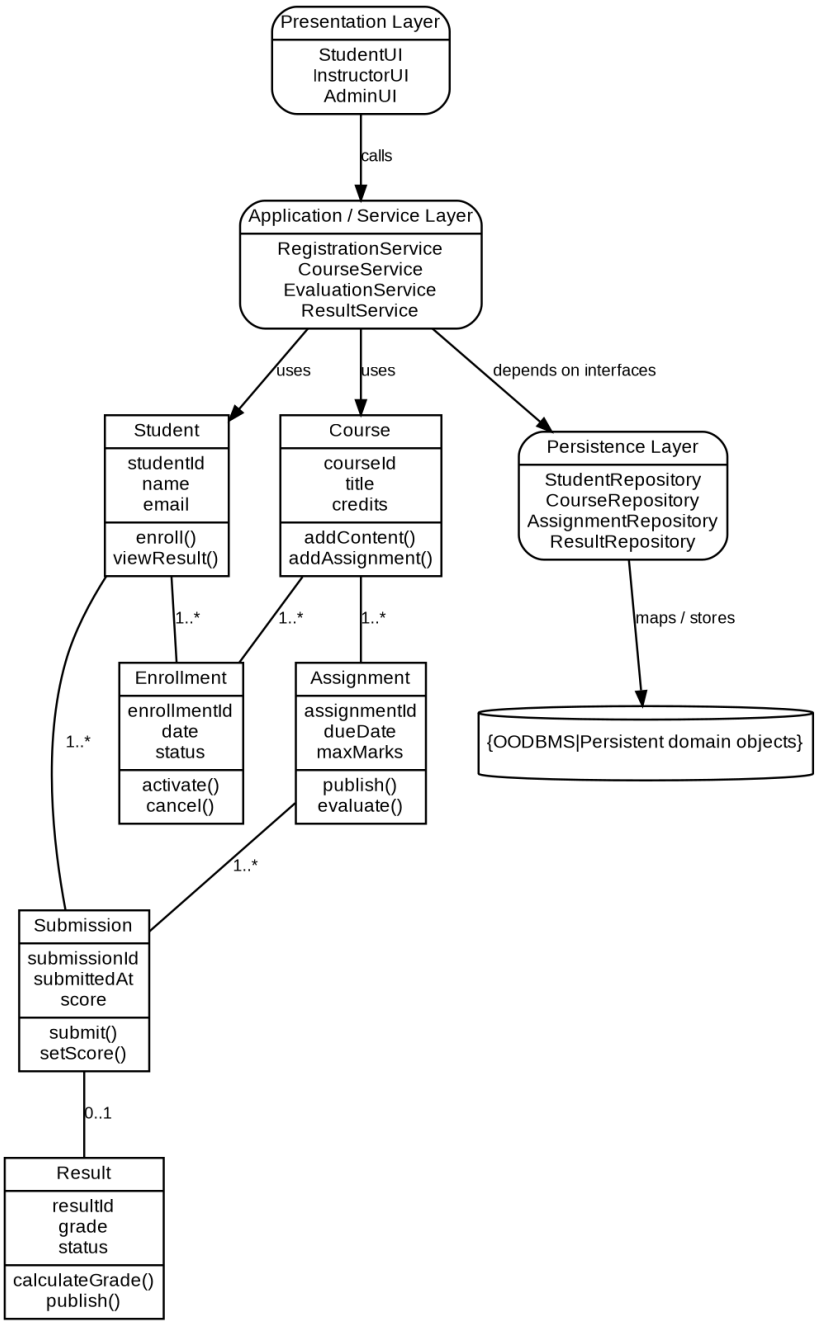

Figure 1: Redesigned UML view for Question 1.

5. Improvements achieved

- Higher cohesion: each class has one focused responsibility instead of mixing UI, rules and storage.
- Lower coupling: UI depends on services; services depend on repository abstractions. Domain classes do not know the database technology.
- Maintainability and testability: services and domain rules can be unit-tested using mock repositories without starting the UI or OODBMS.

- Scalability: new modules such as quizzes, certificates or mobile views can reuse the same service/domain layer; the persistence implementation can also be changed without rewriting the business logic.

---

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

Answer:

1. Reverse-engineered analysis

The repeated if/else or switch statements for email, chatbot, telephone and social media create a rigid design. The dispatcher knows every concrete channel, so adding a channel forces modification of existing code. This violates the Open-Closed Principle and gives the dispatcher more than one responsibility. Channel-specific behavior is also duplicated and difficult to test independently.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods
SupportRequest	Carries the request independently of the channel.	requestId, customerId, message, priority, channel	getPriority(), updateStatus()
SupportChannel interface	Defines the behavioral strategy used to handle a request.	-	handle(request)

EmailChannel	Implements email-specific support behavior.	mailClient	handle(request)
ChatbotChannel	Implements chatbot-specific behavior.	botClient	handle(request)
TelephoneChannel	Implements telephone-support behavior.	telephonyClient	handle(request)
SocialMediaChannel	Implements social-media support behavior.	socialClient	handle(request)
SupportDispatcher	Selects/receives a SupportChannel abstraction and delegates handling.	channelRegistry	dispatch(request)
RequestHandler	Base component for a Chain of Responsibility.	nextHandler	setNext(), process(request)

3. Relationships and redesigned structure

Strategy Pattern is used for channel handling: SupportDispatcher works with the SupportChannel interface, and each channel is a separate interchangeable strategy. A factory or dependency-injection map may choose the strategy from the request channel without placing channel rules in the dispatcher. For common processing steps, a Chain of Responsibility can pass the request through ValidationHandler, PriorityHandler and EscalationHandler.

- 1. A request is created as a SupportRequest.
- 2. Common handlers validate it, assign priority and decide whether escalation is needed.
- 3. SupportDispatcher obtains a SupportChannel implementation and calls handle(request).
- 4. The selected strategy performs only the channel-specific work.

4. UML class / architecture diagram

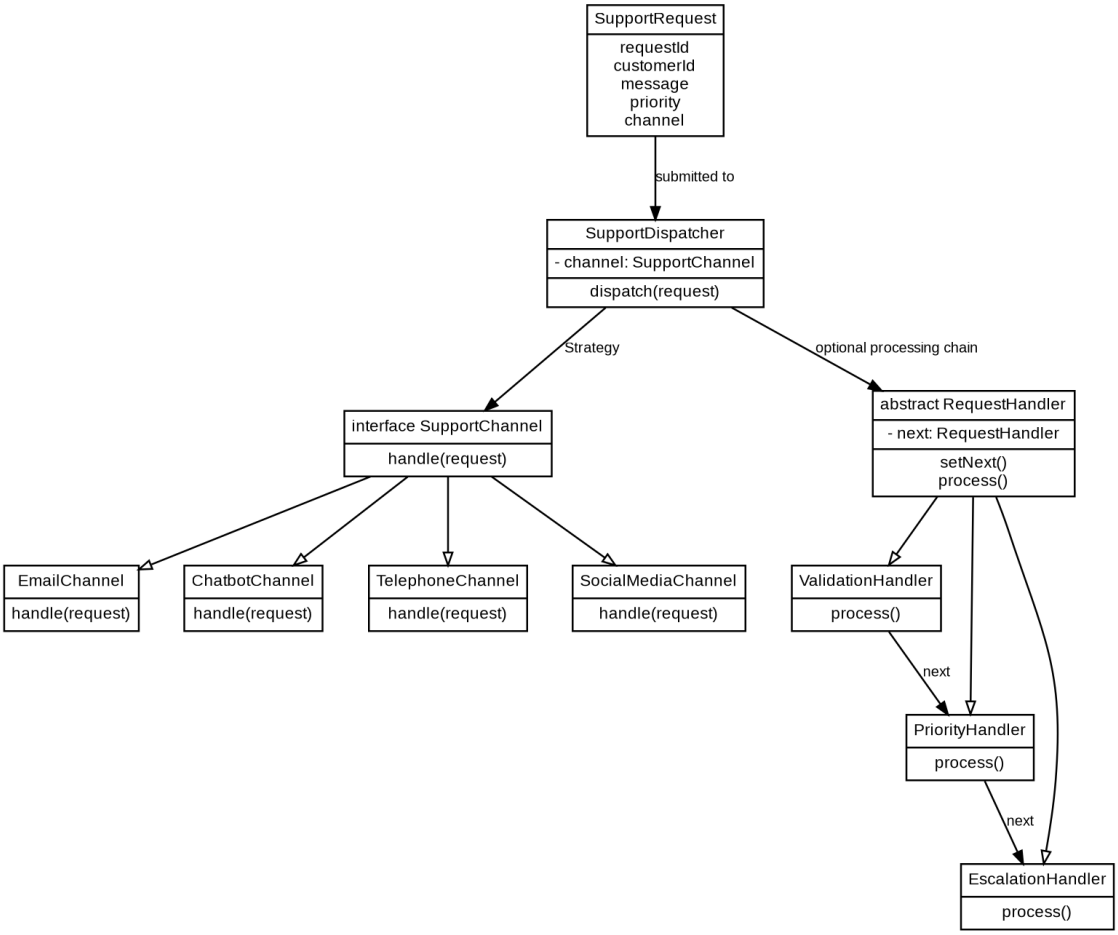

Figure 2: Redesigned UML view for Question 2.

5. Improvements achieved

- Flexibility: channel behavior can be changed at runtime by supplying another SupportChannel strategy.
- Extensibility: adding WhatsAppChannel or VideoSupportChannel requires a new implementation and registration, not modification of all existing modules.
- Loose coupling: dispatcher and business flow depend on an interface rather than email/chatbot/telephone APIs.
- Maintainability and testing: each strategy and each handler can be tested separately; common processing rules are no longer duplicated.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

Answer:

1. Reverse-engineered analysis

In the legacy banking system, account management, validation, transfer, loan processing, reporting and database operations are concentrated in a few classes. Business rules are mixed with SQL/OODBMS calls, which creates low cohesion, high coupling and tests that require the real database.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods
Customer	Bank customer aggregate root.	customerId, name, contact	addAccount(), getAccounts()
Account	Maintains account state and balance rules.	accountNo, balance, type, status	credit(), debit(), canDebit()
Transaction	Represents a monetary transaction.	transactionId, amount, type, timestamp, status	validate(), post(), reverse()
Loan	Represents a loan and its lifecycle.	loanId, principal, rate, tenure, status	calculateEMI(), approve(), reject()

AccountService	Coordinates account use cases.	AccountRepository	openAccount(), closeAccount(), getBalance()
TransferService	Coordinates a transfer as one business operation.	AccountRepository, TransactionRepository, TransactionValidator	transfer(from, to, amount)
LoanService	Coordinates loan use cases.	LoanRepository, CustomerRepository	applyLoan(), approveLoan()
ReportingService	Builds reports from repositories.	TransactionRepository	transactionHistory(), dailySummary()
Repositories / DAOs	Separate persistence from business logic.	AccountRepository, TransactionRepository, LoanRepository, CustomerRepository	findById(), save(), update()

3. Relationships and redesigned structure

A Customer owns one or more Accounts and may have zero or more Loans. An Account owns many Transactions. Service classes coordinate domain objects and use repository interfaces for persistence. Transaction validation is placed behind a TransactionValidator strategy so different validation rules can be substituted without modifying TransferService.

- Presentation layer -> Service layer -> Domain model -> Repository/DAO layer -> Database/OODBMS.
- A Unit of Work or transaction boundary can wrap TransferService.transfer() so debit, credit and transaction logging either all succeed or all roll back.

4. UML class / architecture diagram

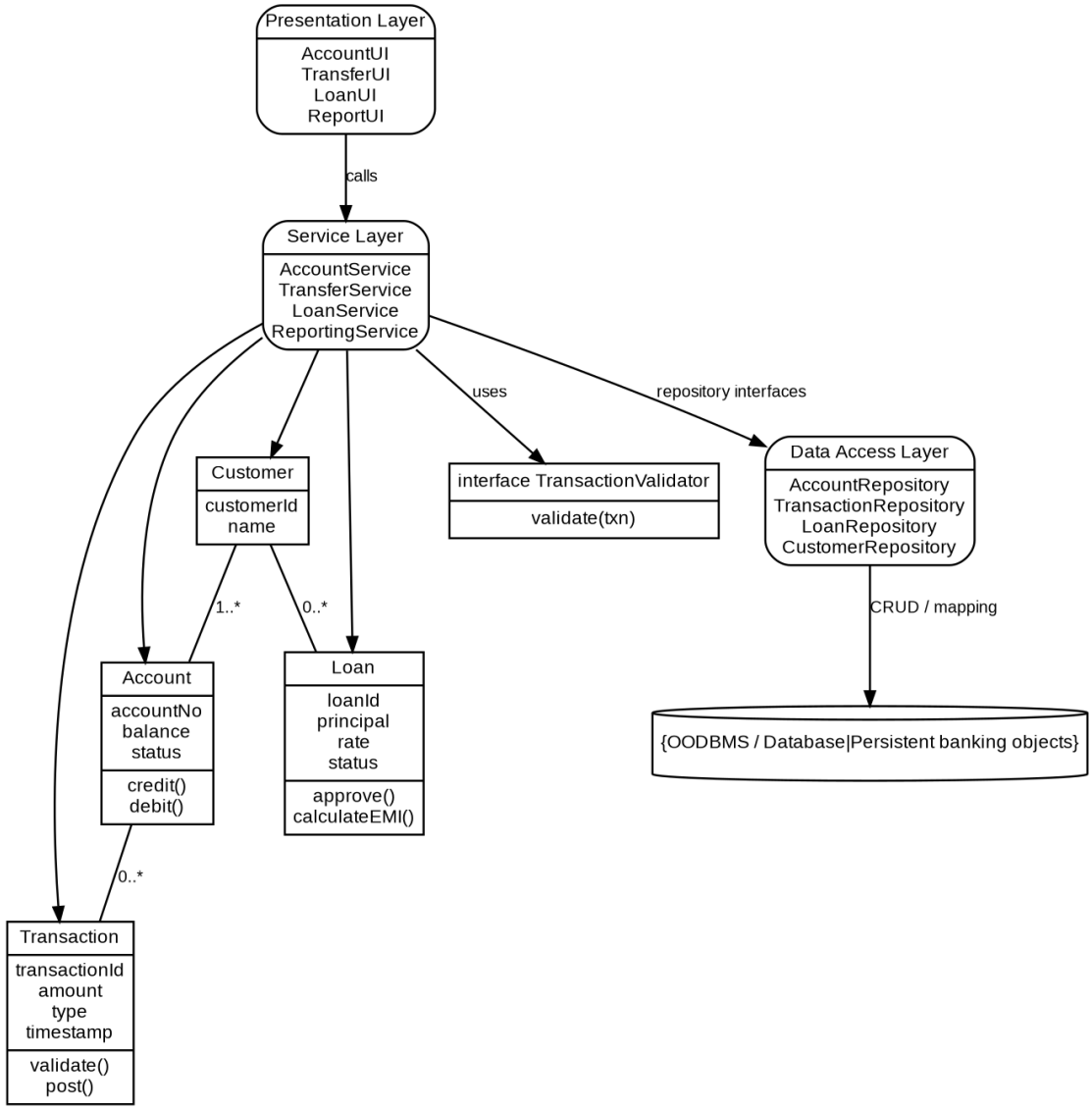

Figure 3: Redesigned UML view for Question 3.

5. Improvements achieved

- Separation of concerns: UI, banking rules and persistence are independent layers.
- Testability: service tests can use fake AccountRepository and TransactionRepository implementations.
- Maintainability: a change to reporting does not change transfer logic, and a database change is isolated to repository implementations.
- Reliability: the service transaction boundary protects consistency during fund transfer, while validation strategies keep rules modular.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

Answer:

1. Reverse-engineered analysis

The reservation screens directly execute OODBMS queries. This tightly couples the user interface to persistence technology, duplicates room/guest/booking/payment queries across modules, and spreads transaction and object-mapping logic throughout the application. A schema or database API change therefore affects many screens.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods
Room	Hotel room domain object.	roomId, type, rate, status	isAvailable(), reserve(), release()
Guest	Guest domain object.	guestId, name, contact	updateProfile()
Booking	Reservation aggregate.	bookingId, checkIn, checkOut, status	confirm(), cancel()
Payment	Payment domain object.	paymentId, amount, mode, status	pay(), refund()
ReservationService	Coordinates search, booking, cancellation and payment.	Repository interfaces	searchRooms(), createBooking(), cancelBooking(), makePayment()

RoomRepository	Abstracts Room persistence.	-	findAvailable(), findById(), save()
BookingRepository	Abstracts Booking persistence.	-	findById(), save(), delete()
GuestRepository	Abstracts Guest persistence.	-	findById(), save()
PaymentRepository	Abstracts Payment persistence.	-	save(), findByBooking()
OODBMS repositories	Concrete data access implementations.	OODB session / connection	map objects, query, persist

3. Relationships and redesigned structure

ReservationScreen depends only on ReservationService. ReservationService uses repository interfaces. Concrete OODBRoomRepository, OODBBookingRepository, OODBGuestRepository and OODBPaymentRepository implement those interfaces. A Guest can have many Bookings; each Booking refers to a Room and may have one Payment.

- DAO/Repository pattern centralizes all persistence operations. Domain objects remain independent of query syntax. For OODBMS integration, repository implementations persist object graphs directly or translate OODBMS object identifiers to domain identities. A Unit of Work can commit the Booking and Payment changes together when needed.

4. UML class / architecture diagram

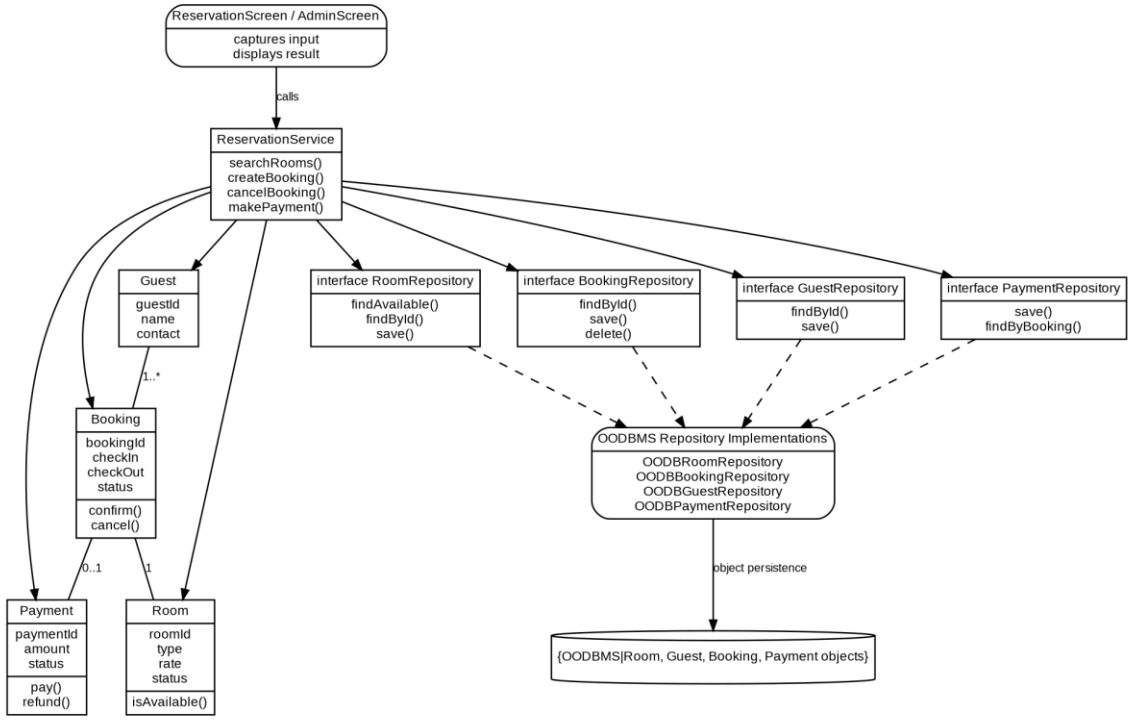

Figure 4: Redesigned UML view for Question 4.

5. Improvements achieved

- Modularity: UI code, reservation rules and OODBMS code can evolve independently.
- Reusability: the same repository methods are reused by reservation, administration and reporting modules instead of repeating queries.
- Reduced coupling and duplication: only repository implementations know the OODBMS API/query language.
- Better integration and testing: repositories provide one controlled persistence boundary and can be replaced with in-memory fakes in tests.

---

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

Answer:

1. Reverse-engineered analysis

The legacy system has separate creation code for Bus, Train, Taxi and RentalVehicle in many modules. Clients must know concrete constructors and repeat the same selection logic. Adding a new transport type causes edits in several places, so object creation is strongly coupled to business code.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods
Transportation interface	Common product abstraction for all transport services.	-	book(), calculateFare(), startService()
Bus	Concrete transportation product.	routeNo, capacity	book(), calculateFare()
Train	Concrete transportation product.	trainNo, coach	book(), calculateFare()
Taxi	Concrete transportation product.	vehicleNo, driver	book(), calculateFare()
RentalVehicle	Concrete transportation product.	vehicleId, ratePerDay	book(), calculateFare()
TransportFactory	Creator abstraction.	-	createTransport(): Transportation

BusFactory / TrainFactory / TaxiFactory / RentalVehicle Factory	Concrete creators.	configuration	createTransport()
BookingServic e	Client that uses abstractions, not constructors.	TransportFactory	bookTrip(), quoteFare()

3. Relationships and redesigned structure

Each concrete transport implements Transportation. Each concrete factory overrides createTransport() and constructs the matching product. BookingService receives a TransportFactory and uses the returned Transportation interface; therefore the client does not contain new Bus(), new Train(), new Taxi() or new RentalVehicle() statements.

- Factory Method is suitable when each service has a distinct creation class. If the system must create families of related objects (for example Vehicle + Ticket + PricingPolicy for each transport mode), the design can be extended to Abstract Factory so one factory creates the whole compatible family.

4. UML class / architecture diagram

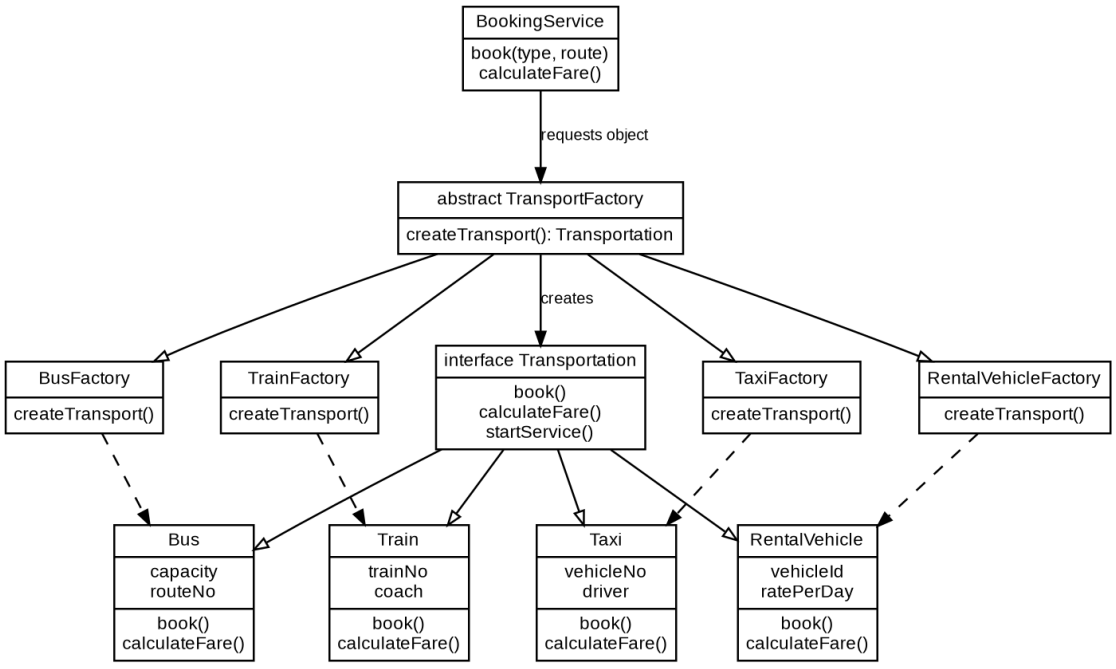

Figure 5: Redesigned UML view for Question 5.

5. Improvements achieved

- Extensibility: a Ferry or Metro is added by implementing Transportation and adding its factory, without rewriting existing booking logic.
- Scalability: object configuration, dependency construction and service-specific setup are centralized in factories.
- Code reuse: common operations are defined by Transportation and common creation rules can stay in the abstract factory.
- Loose coupling: clients program to Transportation and TransportFactory abstractions instead of concrete classes.

---

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

Answer:

1. Reverse-engineered analysis

SRP is violated because the same classes collect patient data, process readings, create alerts, store records and generate reports. OCP is violated because support for a new monitoring device or alert rule requires editing existing conditional logic. DIP is violated because high-level monitoring logic directly creates or calls concrete sensors and storage classes.

2. Proposed classes and responsibilities

Class / Component	Responsibility	Key attributes	Key methods

Patient	Stores patient identity and monitoring profile.	patientId, name, thresholdProfile	updateProfile()
SensorReading	Value object for a captured measurement.	readingId, type, value, timestamp	-
MonitoringDevice interface	Abstracts any sensor/device.	-	read(): SensorReading
HeartRateSensor / GlucoseSensor	Concrete device adapters.	device-specific fields	read()
MonitoringService	Collects and coordinates readings only.	MonitoringDevice list, MedicalRecordRepository, AlertService	collectReadings(), processReading()
AlertRule interface	Strategy for deciding whether a reading is critical.	-	isCritical(reading)
AlertService	Evaluates readings and creates alerts.	AlertRule list, AlertNotifier list	evaluate(), raiseAlert()
AlertNotifier interface	Abstracts notification mechanisms.	-	notify(alert)
MedicalRecordRepository interface	Abstracts storage.	-	saveReading(), findHistory()
OODBMedicalRecordRepository	OODBMS persistence implementation.	OODB session	saveReading(), findHistory()
ReportService	Generates reports from stored history.	MedicalRecordRepository	generateReport(patientId)

3. Relationships and redesigned structure

MonitoringService depends on MonitoringDevice and MedicalRecordRepository interfaces, satisfying DIP. New sensors implement MonitoringDevice and are injected without editing the service, satisfying OCP. AlertRule uses Strategy so new medical rules can be added

independently. AlertNotifier can be used as an Observer-style notification boundary for SMS, dashboard or nurse-station listeners.

- SRP: data collection, alert evaluation, persistence and reporting are separate services.
- OCP: new MonitoringDevice, AlertRule, AlertNotifier or repository implementations are added through interfaces.
- DIP: high-level services depend on abstractions and receive concrete implementations through constructor/dependency injection.

4. UML class / architecture diagram

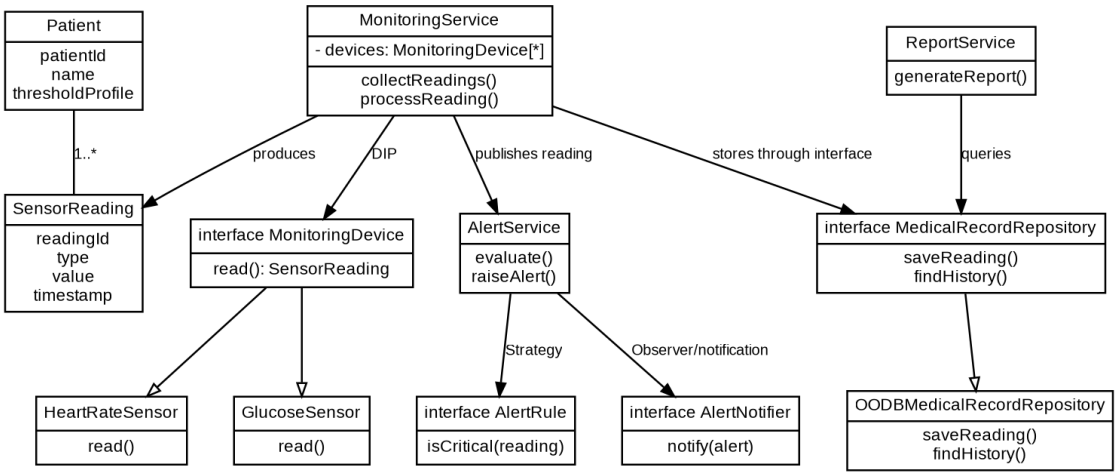

Figure 6: Redesigned UML view for Question 6.

5. Improvements achieved

- Cohesion: each service has one reason to change.
- Loose coupling and flexibility: device vendors, notification methods and OODBMS implementations can change without altering monitoring rules.
- Testability: mock devices can generate deterministic readings; fake repositories can test monitoring and reporting without a real OODBMS.
- Maintainability: smaller interfaces and replaceable strategies localize change and make failures easier to isolate.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation